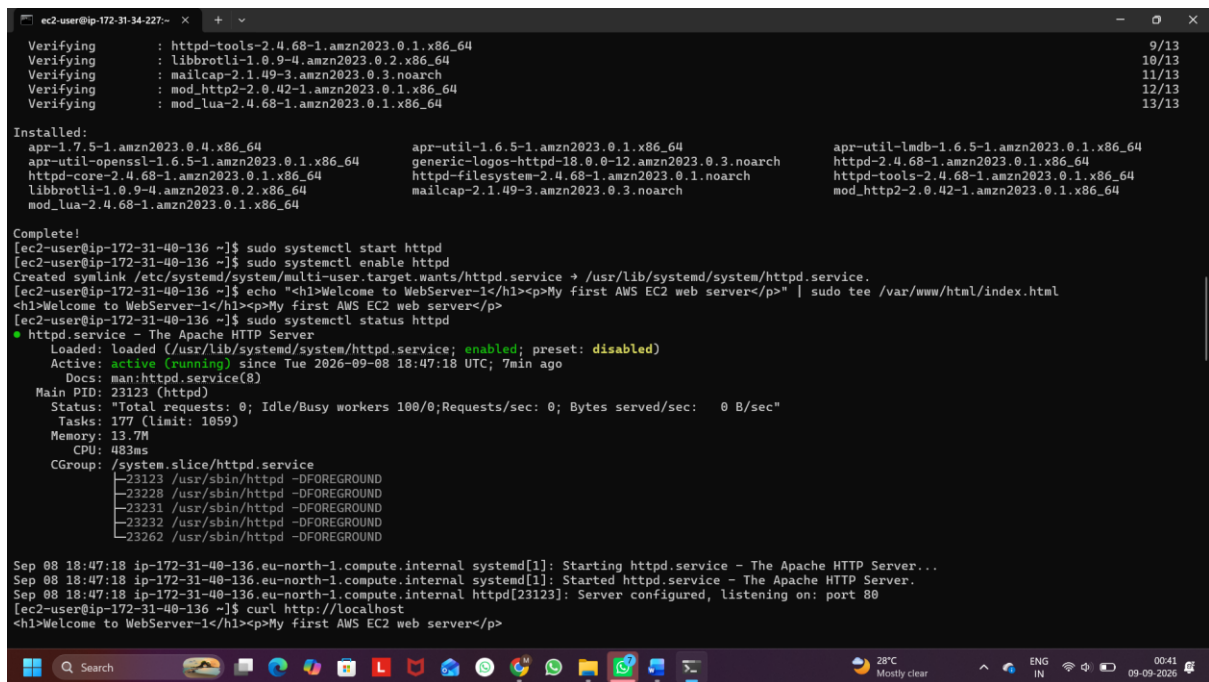


Cloud-Networking -Assignment-2

Q1. In the AWS Management Console, launch two EC2 instances running a simple web server (use Amazon Linux 2 and install httpd or nginx), and verify both are accessible via their public IPs.

Ans:



```
ec2-user@ip-172-31-34-227:~  
Verifying : httpd-tools-2.4.68-1.amzn2023.0.1.x86_64 9/13  
Verifying : libbrotli-1.0.9-4.amzn2023.0.2.x86_64 10/13  
Verifying : mailcap-2.1.49-3.amzn2023.0.3.noarch 11/13  
Verifying : mod_http2-2.0.42-1.amzn2023.0.1.x86_64 12/13  
Verifying : mod_lua-2.4.68-1.amzn2023.0.1.x86_64 13/13  
  
Installed:  
apr-1.7.5-1.amzn2023.0.4.x86_64 apr-util-1.6.5-1.amzn2023.0.1.x86_64 apr-util-ldap-1.6.5-1.amzn2023.0.1.x86_64  
apr-util-openssl-1.6.5-1.amzn2023.0.1.x86_64 generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch httpd-2.4.68-1.amzn2023.0.1.x86_64  
httpd-core-2.4.68-1.amzn2023.0.1.x86_64 httpd-filesystem-2.4.68-1.amzn2023.0.1.noarch httpd-tools-2.4.68-1.amzn2023.0.1.x86_64  
libbrotli-1.0.9-4.amzn2023.0.2.x86_64 mailcap-2.1.49-3.amzn2023.0.3.noarch mod_http2-2.0.42-1.amzn2023.0.1.x86_64  
mod_lua-2.4.68-1.amzn2023.0.1.x86_64  
  
Complete!  
[ec2-user@ip-172-31-40-136 ~]$ sudo systemctl start httpd  
[ec2-user@ip-172-31-40-136 ~]$ sudo systemctl enable httpd  
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.  
[ec2-user@ip-172-31-40-136 ~]$ echo "<h1>Welcome to WebServer-1</h1><p>My first AWS EC2 web server</p>" | sudo tee /var/www/html/index.html  
<h1>Welcome to WebServer-1</h1><p>My first AWS EC2 web server</p>  
[ec2-user@ip-172-31-40-136 ~]$ sudo systemctl status httpd  
● httpd.service - The Apache HTTP Server  
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)  
   Active: active (running) since Tue 2026-09-09 18:47:18 UTC; 7min ago  
     Docs: man:httpd.service(8)  
   Main PID: 23123 (httpd)  
    Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"  
   Tasks: 177 (limit: 1059)  
  Memory: 13.7M  
     CPU: 483ms  
   CGroup: /system.slice/httpd.service  
           └─23123 /usr/sbin/httpd -DFOREGROUND  
             └─23228 /usr/sbin/httpd -DFOREGROUND  
               └─23231 /usr/sbin/httpd -DFOREGROUND  
                 └─23232 /usr/sbin/httpd -DFOREGROUND  
                   └─23262 /usr/sbin/httpd -DFOREGROUND  
  
Sep 08 18:47:18 ip-172-31-40-136.eu-north-1.compute.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...  
Sep 08 18:47:18 ip-172-31-40-136.eu-north-1.compute.internal systemd[1]: Started httpd.service - The Apache HTTP Server.  
Sep 08 18:47:18 ip-172-31-40-136.eu-north-1.compute.internal httpd[23123]: Server configured, listening on: port 80  
[ec2-user@ip-172-31-40-136 ~]$ curl http://localhost  
<h1>Welcome to WebServer-1</h1><p>My first AWS EC2 web server</p>
```

Figure 1: Apache Web Server Running on WebServer-1

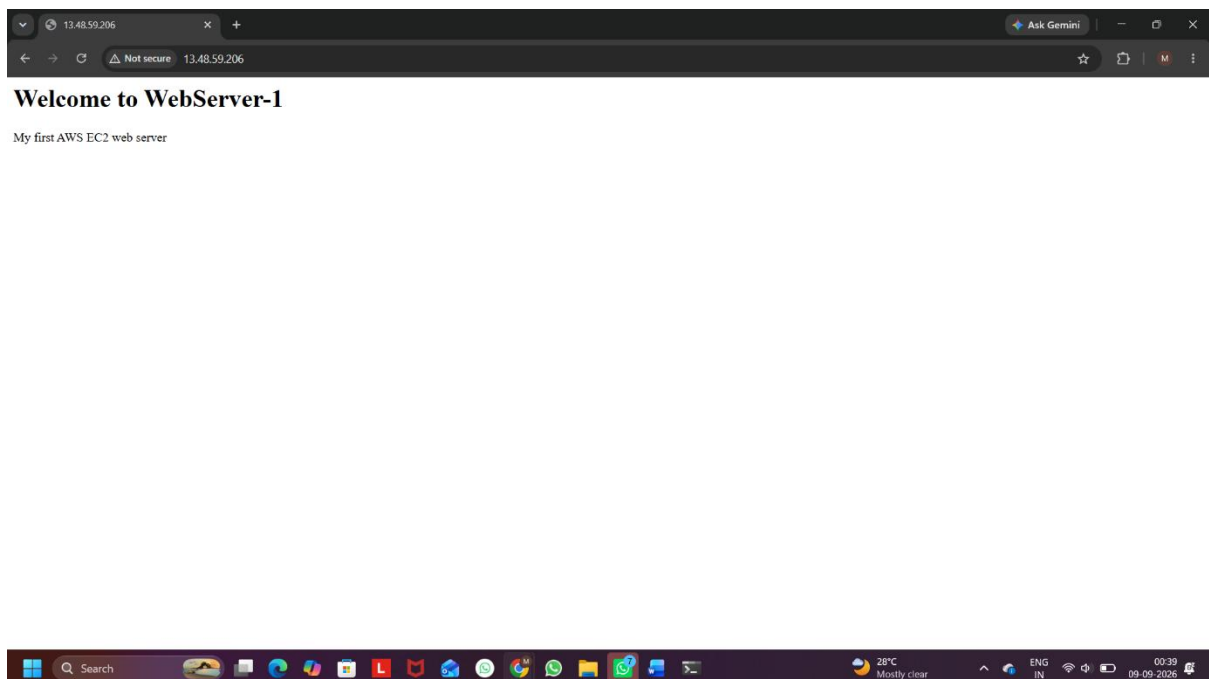


Figure 2: WebServer-1 Website Successfully Accessed Using Public IP

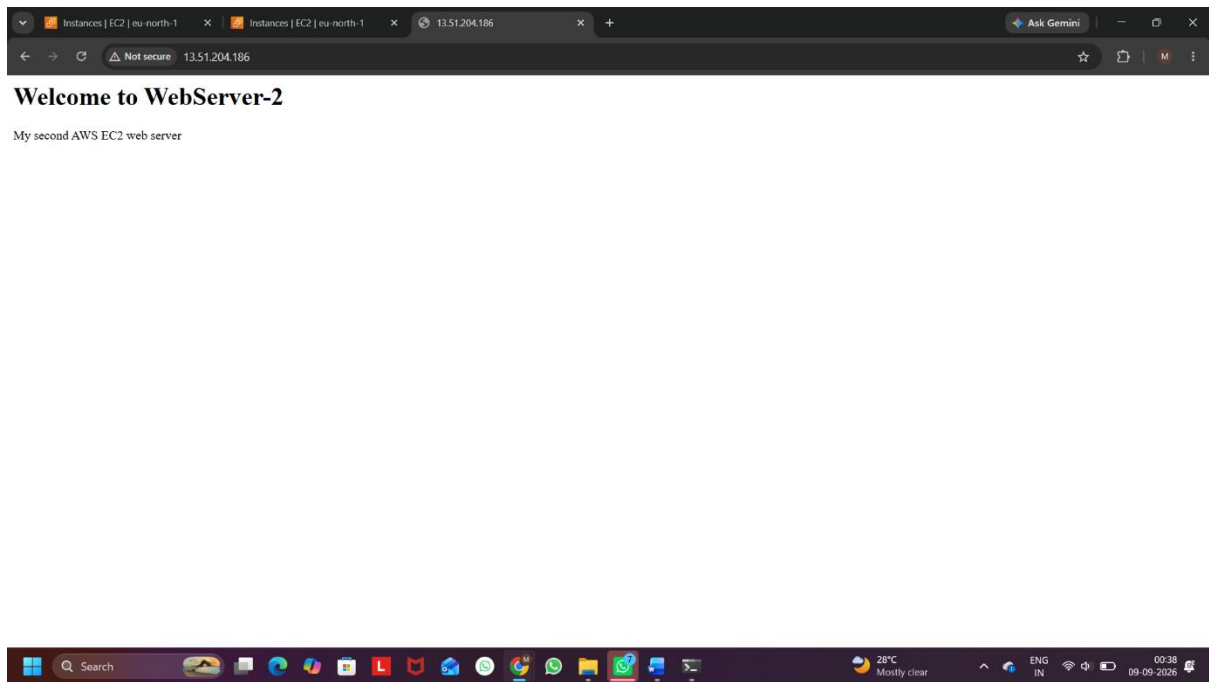


Figure 3: WebServer-2 Website Successfully Accessed Using Public IP

Q2.Set up an Application Load Balancer (ALB) in AWS that distributes incoming HTTP traffic across your two EC2 instances, and test by accessing the ALB's DNS name in your browser to see traffic served from both servers.

Ans:

Verify Target Group Health

The two EC2 instances were registered in the Target Group. After the health checks were completed, both instances showed **Healthy** status.

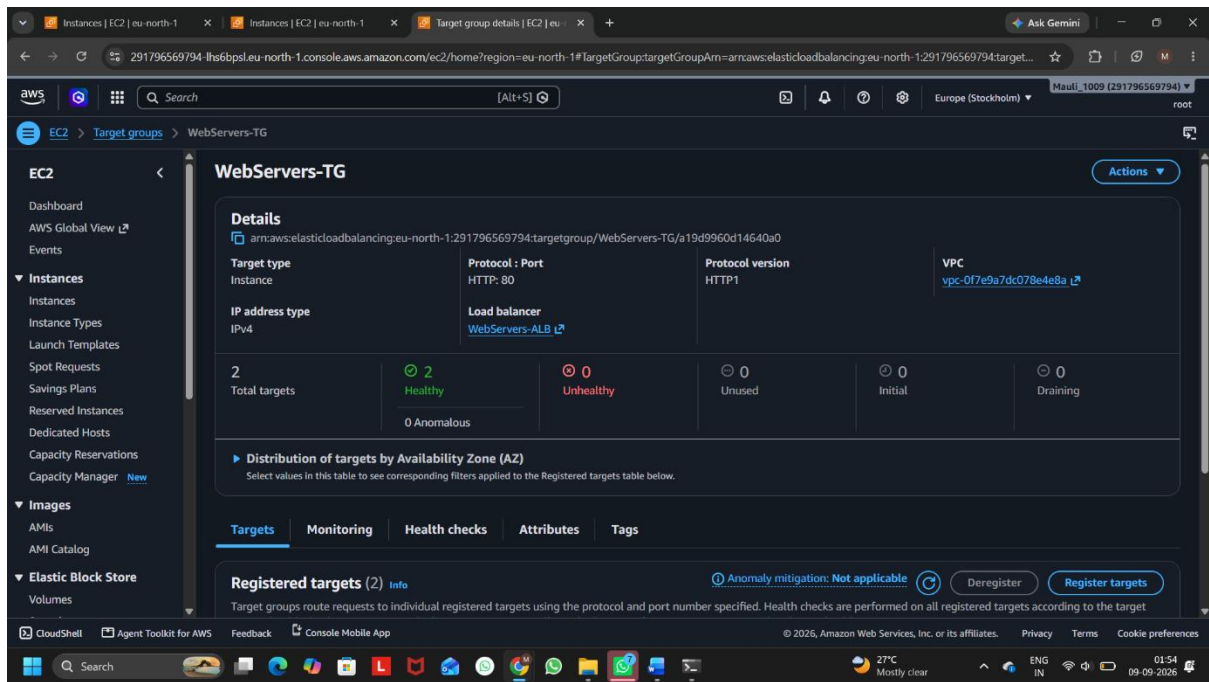


Figure 1: Target Group – Two Healthy EC2 Instances

Test the ALB using its DNS Name

The ALB's DNS name was opened in a web browser. The browser displayed the response from WebServer-1, showing that the ALB successfully forwarded the HTTP request to the first EC2 instance.

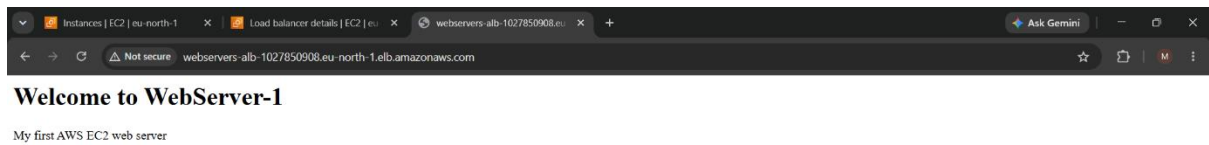


Figure 2: ALB DNS – Response from WebServer-1

After refreshing the same **ALB DNS name**, the browser displayed the response from **WebServer-2**. This confirms that the ALB distributed the incoming HTTP traffic to the second EC2 instance as well.

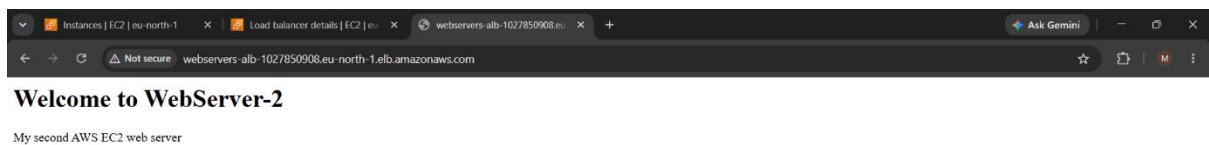


Figure 3: ALB DNS – Response from WebServer-2

Result: The ALB successfully distributed HTTP traffic across both EC2 instances.

Q3. Change the health check settings of your ALB to use a custom path (e.g., /health) and verify that the load balancer only routes traffic to healthy instances. Hint: Create a /health endpoint on your web server that returns a 200 OK response.

Ans:

Aim

To change the ALB health check path from / to a custom path **/health** and verify that the ALB routes traffic only to **healthy EC2 instances**.

Procedure

First, a custom /health endpoint was created on both EC2 web servers. This endpoint returns a successful HTTP 200 OK response when the web server is running.

Next, the Target Group health check settings were opened and the default health check path / was changed to the custom path:

/health

The ALB then started using /health to check whether the EC2 instances were healthy.

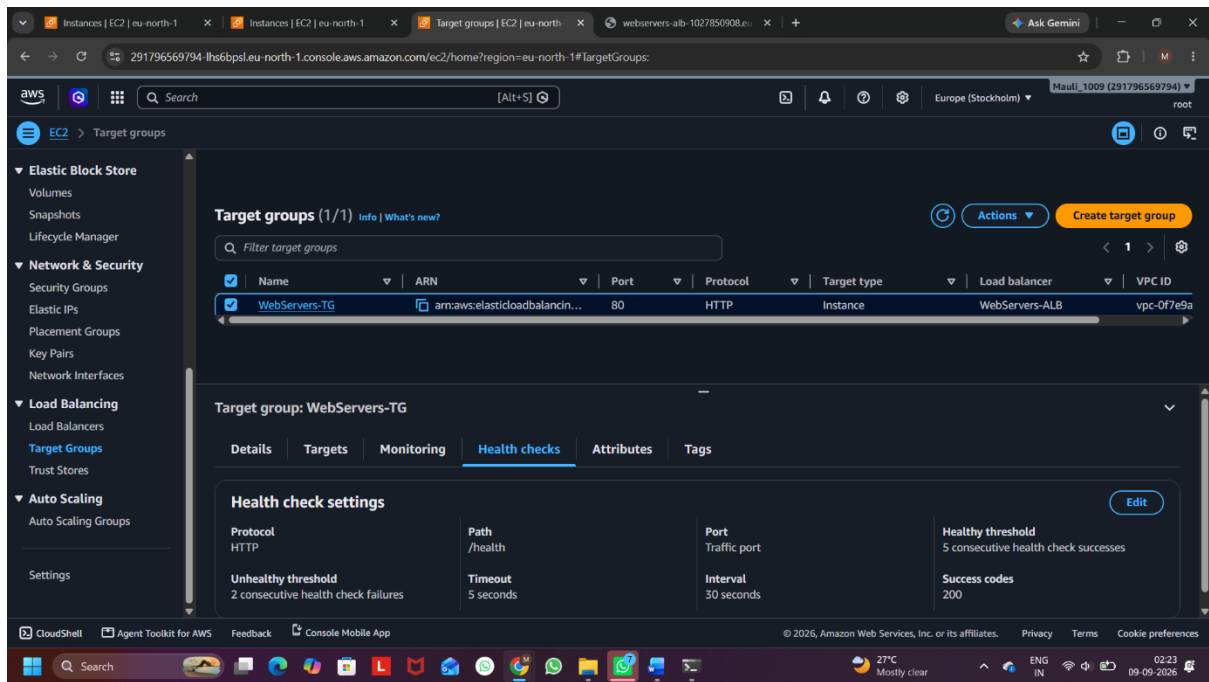


Figure 1: Custom /health Health Check

After that, to test the health check functionality, the web server on WebServer-2 was temporarily stopped. Since WebServer-2 could no longer provide a successful response from /health, the ALB marked it as Unhealthy. WebServer-1 was still running and therefore remained Healthy.

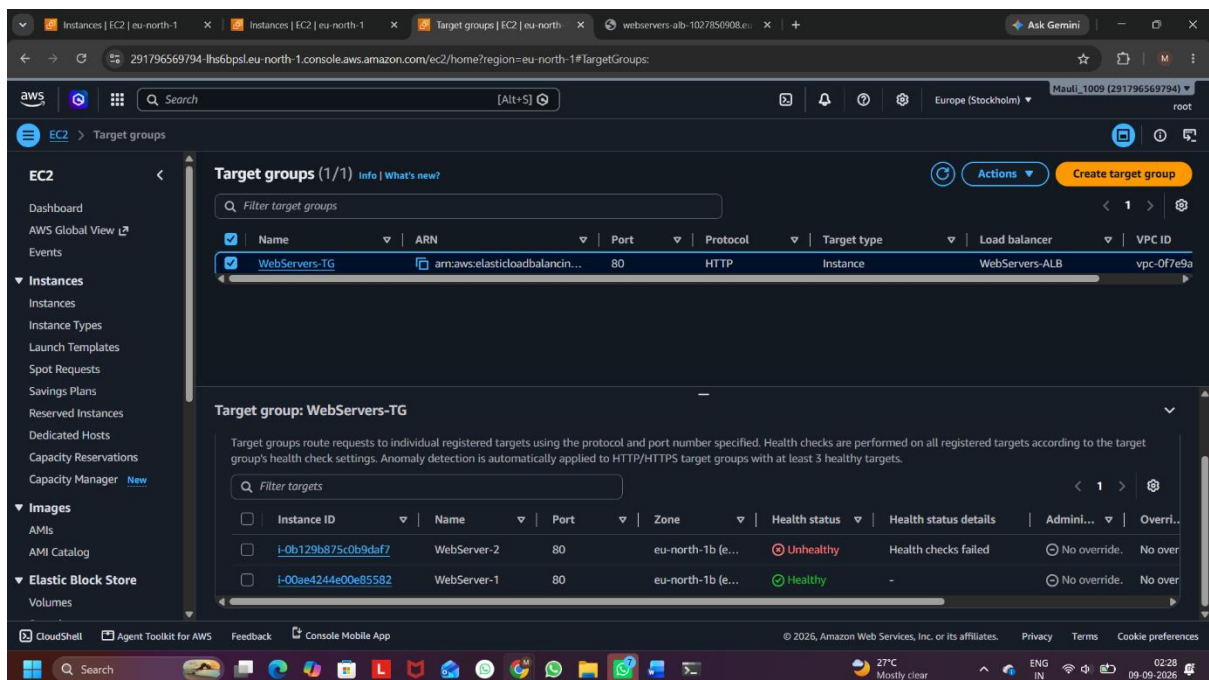


Figure 2: One Healthy and One Unhealthy Target

Finally, the ALB DNS name was opened in the browser. Since WebServer-1 was healthy and WebServer-2 was unhealthy, the ALB forwarded the request only to WebServer-1. Even after refreshing the page, the response from WebServer-1 was displayed because the unhealthy WebServer-2 was removed from traffic routing.

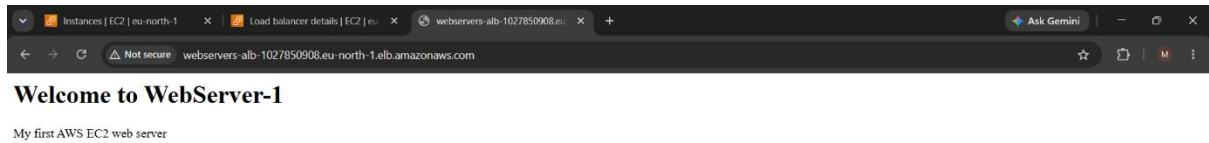


Figure 3: ALB Routes Traffic Only to Healthy Server

Result

The ALB health check was successfully changed to the custom /health path. When WebServer-2 became unhealthy, the ALB successfully stopped sending traffic to it and routed requests only to the healthy WebServer-1.

Q4. Simulate a server failure by stopping one of your EC2 instances and observe how the ALB handles requests — describe what happens and how this demonstrates high availability.

Ans:

Aim

To simulate an **EC2 server failure** by stopping one EC2 instance and observe how the **Application Load Balancer (ALB)** handles requests. This demonstrates **High Availability**.

Procedure

First, both EC2 instances were registered with the ALB Target Group and were serving web traffic.

To simulate a server failure, **WebServer-2 EC2 instance was stopped** from the EC2 console.

After stopping the instance, the ALB detected that WebServer-2 was no longer available. WebServer-1 remained **Healthy**, while WebServer-2 showed **Unused** because the EC2 instance was in the stopped state.

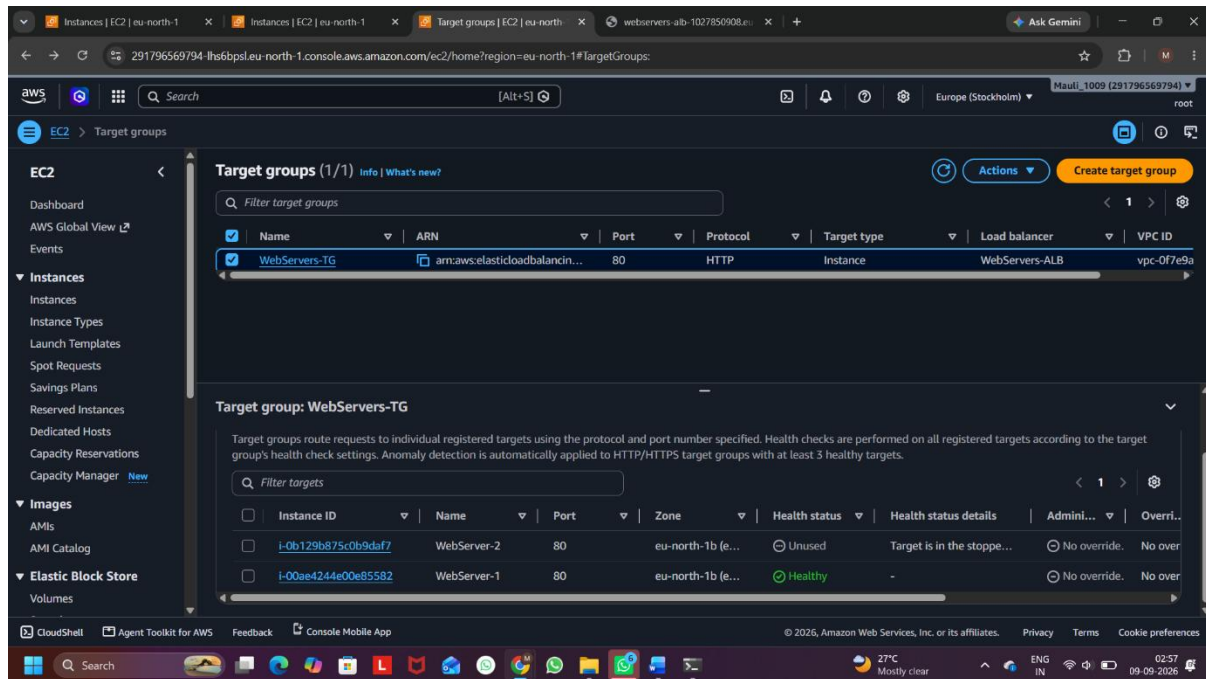


Figure 1: One Healthy Server and One Stopped (Unused)Server

Next, the **ALB DNS name** was accessed in a web browser. The ALB automatically routed the request to the remaining healthy **WebServer-1**. Even after refreshing the page, the application continued to respond from WebServer-1 because WebServer-2 was stopped and was not available to receive traffic.

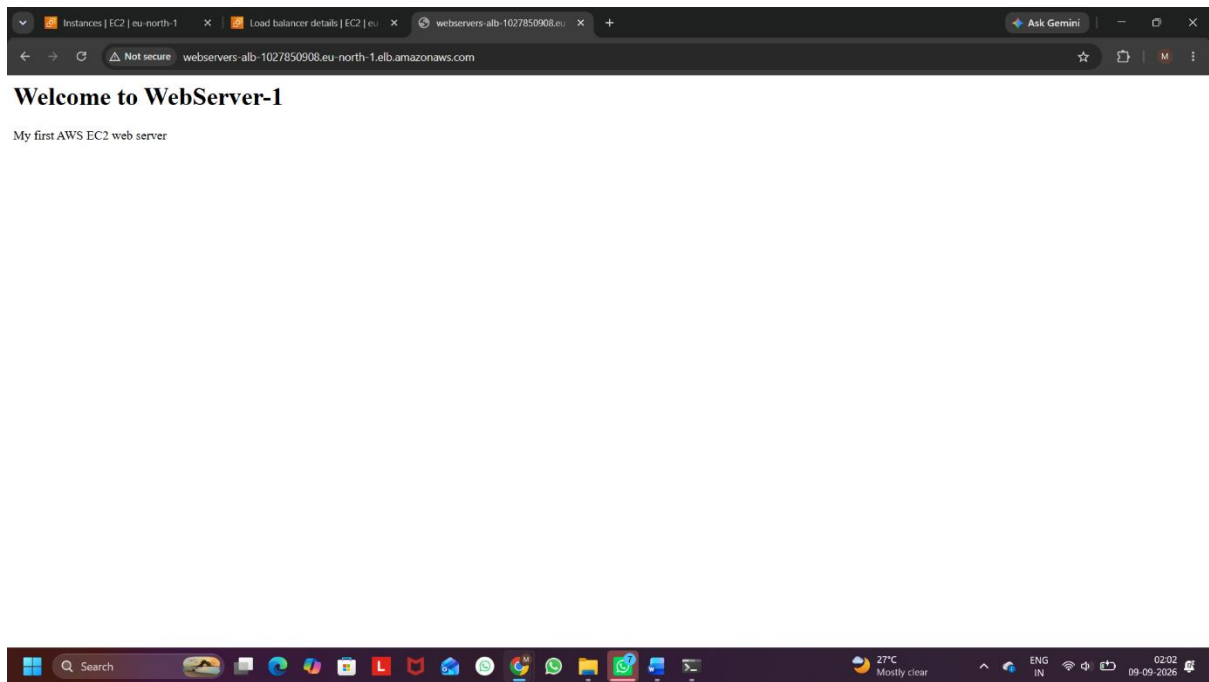


Figure 2: ALB Serving Healthy Server After EC2 Failure

Result

The server failure was successfully simulated by stopping one EC2 instance. The **Application Load Balancer** continued routing traffic to the remaining healthy EC2 instance, proving that the application remained available during a server failure.

Q5.Compare Application Load Balancer, Network Load Balancer, and Classic Load Balancer in a table with at least 3 points each, focusing on supported protocols, use cases, and features.

Ans:`

AWS provides different types of Load Balancers for different types of applications. **Application Load Balancer (ALB)** works mainly at the application layer, **Network Load Balancer (NLB)** works at the network/transport layer, and **Classic Load Balancer (CLB)** is the older generation load balancer.

Point	Application Load Balancer (ALB)	Network Load Balancer (NLB)	Classic Load Balancer (CLB)
Protocols	HTTP, HTTPS	TCP, UDP, TLS, QUIC	HTTP, HTTPS, TCP, SSL

Point	Application Load Balancer (ALB)	Network Load Balancer (NLB)	Classic Load Balancer (CLB)
Layer	Layer 7	Layer 4	Layer 4 & basic Layer 7
Use Case	Web applications, APIs, microservices	High-performance and low-latency applications	Legacy applications
Routing	Path, host, headers, etc.	IP address and port	Basic routing
Key Features	Path-based routing, host-based routing, authentication	Very high performance, low latency, static IP	Basic health checks, SSL termination, stickiness

- **ALB** is mainly used for modern web applications and APIs. **NLB** is used for high-performance and TCP/UDP applications. **CLB** is mainly used for older or legacy applications.